Generative Adversarial Network (GAN) for MNIST Digit Generation

1. Objective

The primary objective of this assignment is to implement and train a Generative Adversarial Network (GAN) using the TensorFlow Keras API. The goal is to generate synthetic grayscale images of handwritten digits that are statistically similar to the MNIST dataset. This project demonstrates the practical application of adversarial learning and generative modelling.

2. Brief Explanation of GANsA :

Generative Adversarial Network (GAN) is a class of machine learning frameworks designed by Ian Goodfellow and his colleagues in 2014. It consists of two neural networks—the Generator and the Discriminator—which are trained simultaneously through a competitive process known as a zero-sum game.

The Generator (G):

This network acts as a "counterfeiter." It takes a random noise vector as input and attempts to transform it into a meaningful data sample, such as a 28x28 digit image. Its goal is to produce images so realistic that the Discriminator cannot distinguish them from actual data.

The Discriminator (D):

This network acts as the "police" or "art critic." It is a binary classifier that receives an image and outputs a probability score representing whether the image is "real" (from the MNIST dataset) or "fake" (created by the Generator).

Adversarial Training:

During training, the Generator is penalized when the Discriminator correctly identifies its output as fake, forcing it to improve its technique. Conversely, the Discriminator is penalized when it is fooled by the Generator or misidentifies a real image. Ideally, this process reaches a "Nash Equilibrium" where the Generator produces perfect replicas and the Discriminator can only guess with 50% accuracy.

3. Architecture Summary

Following the provided design guidelines, the architecture is structured as follows:

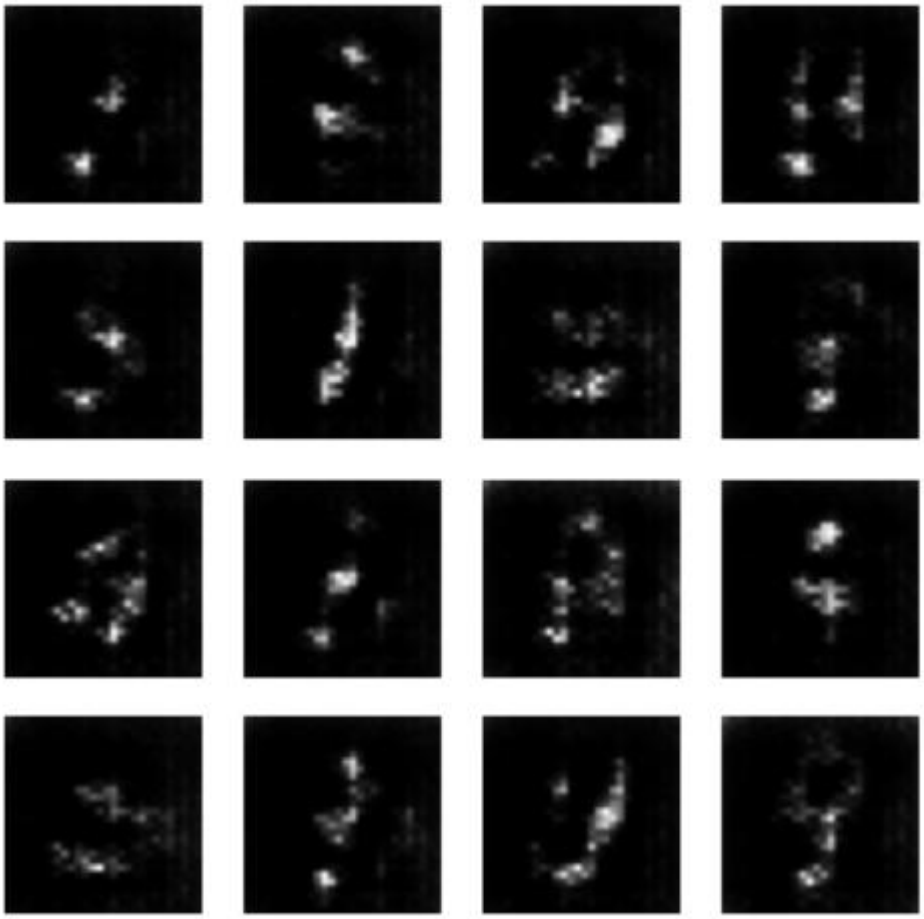
Generator Architecture:

Utilizes a Dense input layer followed by BatchNormalization and LeakyReLU activations. It then uses Conv2DTranspose layers to upscale the noise into a 28x28x1 image, concluding with a Tanh activation to bound the pixel values between -1 and 1.

Discriminator Architecture:

Functions as a convolutional classifier using Conv2D layers and LeakyReLU. Dropout is applied to prevent overfitting and ensure the Discriminator does not become too powerful too quickly. It ends with a Flatten layer and a Sigmoid output to produce a probability score.

Epoch 1 / 20

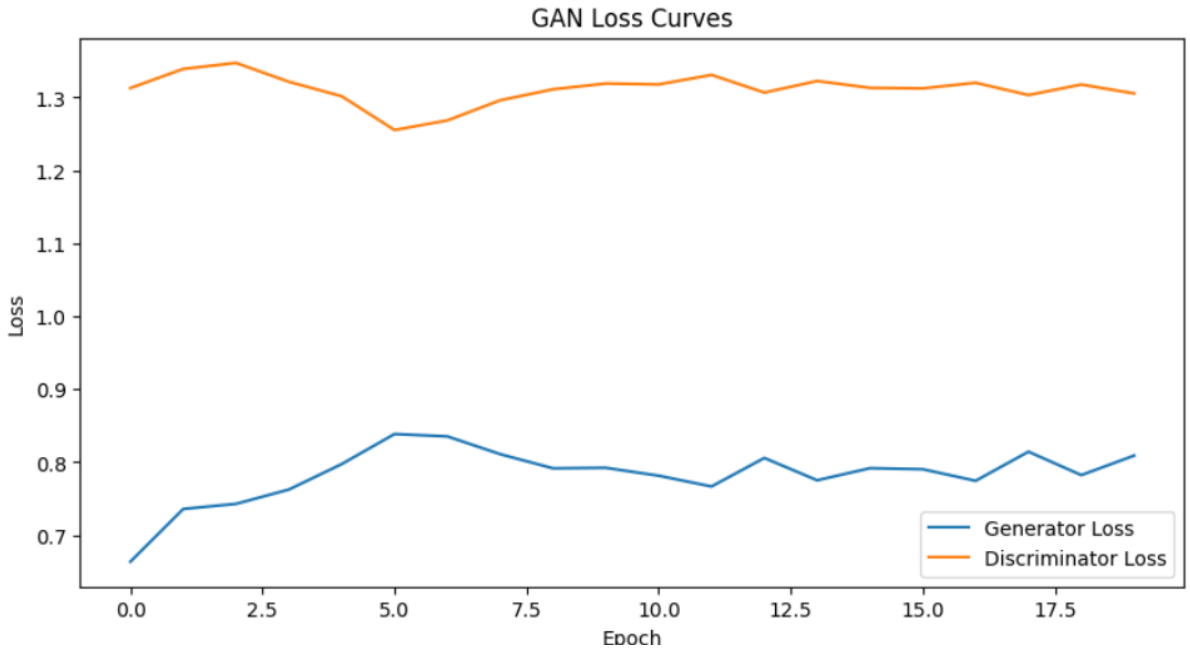


Epoch 10 / 20

Epoch 5 / 20

Epoch 15 / 20

Epoch 20 / 20


GAN Loss Curves
Loss
Epoch
Generator Loss
Discriminator Loss
1.3
1.2
1.1
1.0
0.9
0.8
0.7
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5